Octane Security Report

Security Analysis of Eco: eco-delivery
9/2/2026

About Octane

Octane is a security intelligence platform purpose-built for modern software teams. We help engineering and security teams identify, investigate, and triage vulnerabilities faster—without disrupting the development process.

Combining automated static analysis with human-readable explanations, Octane bridges the gap between high-signal tooling and real-world developer workflows. Our platform provides contextualized vulnerability reports, clustered findings, and AI-enhanced guidance to make security reviews faster, clearer, and more actionable.

Octane was designed in collaboration with top auditors and engineers to meet the unique demands of web3 and next-gen software systems. We work closely with security researchers, protocol teams, and developer-first organizations to continuously improve our threat detection models and surface what actually matters.

Our clients include leading teams across DeFi, L2 protocols, infrastructure tooling, and security research. Octane integrates seamlessly with developer workflows—from GitHub automation to custom feedback tooling—so teams can reduce noise, eliminate risk, and ship with confidence.

Octane is backed by deep domain knowledge in static analysis, program synthesis, and exploit modeling, with capabilities designed for both technical depth and usability.

To learn more or request a demo, visit www.octane.security, or reach out to us at info@octane.security.

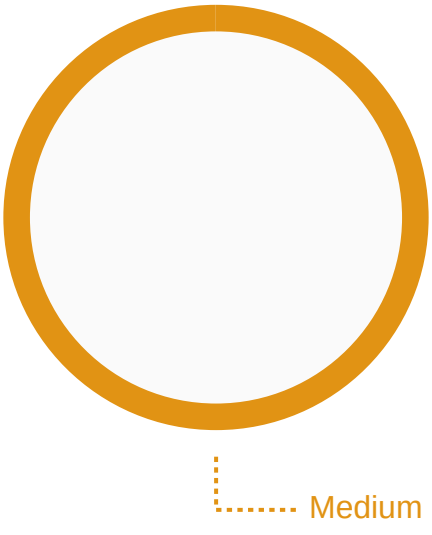
Octane Security
Distributed team with a global presence
www.octane.security

"Summary of findings"

This analysis reviewed the Eco: eco-delivery smart contracts using Octane's automated analysis and included team feedback on findings.

The analysis identified a total of 1 issues. After triage and review, 1 of these issues have been resolved or acknowledged.

Vulnerabilities Confirmed	01/01
Critical	00/00
High	00/00
Medium	01/01
Low	00/00
Informational	00/00



Vulnerabilities & Warnings

Vulnerability #1

Missing memo/Instructions-sysvar support in deliver_token causes DoS for Token-2022 recipients with MemoTransfer

Severity: MEDIUM | Likelihood: LOW | Impact: MEDIUM | Status: Will fix

Confirmed and fixed: https://github.com/eco/eco-delivery/pull/4One correction for analyzer accuracy: the stated mechanism is wrong. Thishas nothing to do with the Instructions sysvar. Token-2022 validates thememo via check_previous_sibling_instruction_is_memo(), which takes noarguments and uses the get_processed_sibling_instruction syscall(spl-token-2022/src/extension/memo_transfer/mod.rs). No account isrequired, and the memo_transfer module contains no sysvar use at all. Theproposed diff does not forward the sysvar either, so the description andthe patch disagree.The conclusion is correct and the DoS reproduces(deliver_token_token2022_memo_required_recipient_is_rejected). Scenario 2is also correct and is the reason this was worth fixing: because the checkonly sees siblings at the same CPI stack height, a top-level memo does notsatisfy a transfer issued from inside deliver_token, so callers cannot workaround it. Pinned bydeliver_token_token2022_top_level_memo_does_not_satisfy_the_check.Fixed with an optional SPL Memo account rather than a mandatory one, sodeliveries that do not need a memo pay nothing. Note this is still abreaking wire change: Anchor encodes an absent optional account as theprogram's own id occupying the slot, so nine-account callers are rejected.

The deliver_token instruction only forwards the four base accounts to Token-2022 transfer_checked and does not emit a memo or pass the Instructions sysvar. For recipients whose Token-2022 ATA requires a memo (MemoTransfer), the token program cannot validate a preceding memo and the transfer consistently fails (e.g., NoMemo), causing persistent DoS for those recipients.

In solana/programs/deliver/src/instructions/deliver_token.rs, handle_deliver_token performs a CPI to token_interface::transfer_checked with only [from, mint, to, authority] and does not include any remaining accounts or emit a memo. Token-2022's MemoTransfer extension requires a memo before incoming transfers and validates it by inspecting the prior top-level instruction; in practice, the token program must be able to read the Instructions sysvar or the immediately preceding sibling instruction at the correct stack height. Because deliver does not forward the Instructions sysvar and does not issue a memo immediately before the transfer CPI, transfers into a recipient ATA with MemoTransfer enabled revert (e.g., with NoMemo). A top-level memo placed before the outer instruction does not satisfy the nested CPI's memo check. This produces a consistent denial

of service for deliveries to memo-required Token-2022 accounts. The issue does not affect recipients whose ATAs do not require memos; if the recipient ATA is created by deliver, it does not have MemoTransfer enabled by default and the transfer succeeds.

file: solana/programs/deliver/src/instructions/deliver_token.rs

```
 4    anchor_spl::{
 5    associated_token::AssociatedToken,

 6    token_interface::{transfer_checked, Mint, TokenAccount,
     TokenInterface, TransferChecked},
 7    },

...

65    pub associated_token_program: Program<'info, AssociatedToken>,
66    pub system_program: Program<'info, System>,

67   }

...

93    let vault_seeds: &[&[u8]] = &[VAULT_SEED, &[bump]];
94


95    transfer_checked(
96    CpiContext::new_with_signer(
```

```
 4    anchor_spl::{
 5    associated_token::AssociatedToken,
 6  + memo::{self, Memo},
 7    token_interface::{transfer_checked, Mint, TokenAccount,
     TokenInterface, TransferChecked},
 8    },

...

66    pub associated_token_program: Program<'info, AssociatedToken>,
67    pub system_program: Program<'info, System>,
68  + pub memo_program: Program<'info, Memo>,
69    }
```

```
...

95    let vault_seeds: &[&[u8]] = &[VAULT_SEED, &[bump]];
96
97  +// Emit a memo immediately before the transfer to satisfy
     Token-2022 MemoTransfer recipients.
98  + memo::build_memo(CpiContext::new(ctx.accounts.memo_pro-
     gram.to_account_info(), memo::BuildMemo {}), b"deliver")?;
99    transfer_checked(
100   CpiContext::new_with_signer(
```

Severity

Impact Explanation: [Medium] Consistent denial of service of the core delivery function for recipients whose Token-2022 accounts require memos; routes revert until the requirement is disabled or the program is updated.

Likelihood Explanation: [Low] Requires a specific but legitimate recipient account policy (MemoTransfer enabled on the existing ATA), which is uncommon though plausible; once present, any standard call triggers the failure.

Exploitation

Exploitation Scenarios:

Scenario 1.
Direct call: A user calls deliver_token to sweep the vault's Token-2022 balance to a recipient whose existing ATA has MemoTransfer enabled. The instruction passes the min check, then issues transfer_checked without the Instructions sysvar or an immediately preceding memo at the correct stack level. The Token-2022 program fails the transfer (e.g., NoMemo), and the transaction reverts, preventing delivery.
Preconditions / Assumptions
(a). A Token-2022 mint exists.
(b). The vault's ATA for the mint holds a positive balance (amount >= min).
(c). The recipient's ATA for the mint already exists and has MemoTransfer enabled (requires incoming memos).
(d). deliver_token is invoked with token_program = Token-2022 and min = min).
(c). The recipient's ATA for the mint already exists and has MemoTransfer enabled (requires incoming memos).
(d). The route is executed under a single top-level Portal.fulfill instruction; a top-level Memo precedes Portal.fulfill.
(e). deliver_token is invoked as a nested CPI and still does not emit a memo or forward the Instructions sysvar to the Token-2022 CPI.

Warning #1

Sweep-all transfer pattern in Deliver.deliverToken for sender-surcharge ERC-20s causes per-asset DoS

Severity: MEDIUM | Likelihood: MEDIUM | Impact: MEDIUM | Status: Acknowledged

Accepted as a documentation issue; no code change. Confirmed by test. All three claims reproduce against a sender-surcharge mock: - safeTransfer(recipient, balance) reverts, because the contract holds exactly `balance` and the token debits balance + fee(balance). - Topping the contract up does not help. The sweep then attempts the new, larger balance and falls short by the fee on that larger amount. There is no balance at which it starts working. This is the non-obvious part of the finding and the main reason it was worth acting on. - A zero balance still no-ops, since fee(0) == 0. Two points on severity. The failure is fail-closed: the transfer reverts,Two points on severity. The failure is fail-closed: the transfer reverts,the route reverts atomically, and nothing is lost -- the asset isundeliverable, not unsafe. And a token with these semantics is broken wellbeyond this contract: no holder of one can ever transfer their entire-balance to anyone, so every exact-balance sweep in DeFi fails on it. Wecould not identify a deployed token with this behaviour; the scenarios werereproducible only against a mock written for the purpose.We have documented the incompatibility rather than working around it, whichis the same treatment given to other unsupported token classes (Token-2022TransferHook mints on the SVM side). Deliver.sol is immutable and deployedto 13 mainnets, so a behavioural change is not available in any case.Scoped smaller than proposed: the note is folded into the existingfee-on-transfer WARNING, which already covers tokens that alter transferamounts but previously described only the under-delivery direction. This isthe missing half of the same warning, and it sits where an integratorevaluating an unusual token is already reading. A separate section wouldhave pushed the load-bearing warnings (atomicity, address(0), thepre-transfer min check) further down.

Deliver.deliverToken always transfers its full ERC-20 balance. For ERC-20s that debit amount+fee from the sender (sender-surcharge), this call consistently reverts because the contract never holds amount+fee. As a result, any atomic route ending with such an asset is persistently unusable via Deliver (per-asset DoS), and solvers may incur wasted gas until they filter the asset.

In evm/src/Deliver.sol, deliverToken reads the contract's pre-transfer token balance B, checks B >= min, and calls safeTransfer(recipient, B). For ERC-20s that charge a sender-surcharge (debiting amount + fee(amount) from the sender to guarantee the recipient receives amount), safeTransfer(recipient, B) requires

B + fee(B) but only B is available, so the token reverts. This failure is deterministic for any positive fee and any nonzero balance, and topping up does not help because the function always attempts to send the new full balance, again lacking the extra fee. In correct atomic routes, the revert rolls back the entire transaction, so user funds are not lost; however, the output asset becomes undeliverable through this contract (persistent per-asset DoS). Additionally, executors/solvers can suffer gas griefing while discovering and filtering such assets. This arises from the intentional sweep-all design and affects only ERC-20s with sender-surcharge semantics.

file: evm/src/Deliver.sol

```
82   ///      this contract at all.
83   ///
84   ///      ## WARNING — `address(0)` is a native-ETH sentinel, not a rejected input
85   ///
...
165   if (balance < min) revert BalanceBelowMin(balance, min);
166
167   token.safeTransfer(recipient, balance);
168  }
```

```
82   ///      this contract at all.
83   ///
84  +///     ## Incompatibility — sender-surcharge tokens (fee charged on top)
85  +///
86  +///      Some ERC-20s debit `amount + fee(amount)` from the sender (to preserve the recipient's full
87  +///      `amount`). Because this contract always attempts to transfer its entire balance, such tokens
88  +
```

```
    ///      will revert on every outbound transfer here. Integrators
    must not route those tokens through
89  +///      this contract; filter or simulate them upstream.
90  +///
91   ///      ## WARNING — `address(0)` is a native-ETH sentinel, not
    a rejected input
92   ///

...

172   if (balance < min) revert BalanceBelowMin(balance, min);
173
174 +// NOTE: Sender-surcharge (amount+fee) ERC-20s are
    incompatible: attempting to transfer the
175 +// entire balance requires more than is available and will
    revert. Filter/simulate upstream.
176   token.safeTransfer(recipient, balance);
177  }
```

Severity

Impact Explanation: [Medium] Delivery is a core function; for affected tokens it is persistently unavailable, causing significant but limited (per-asset) DoS. No protocol-wide failure or frozen funds in atomic paths.

Likelihood Explanation: [Medium] Requires a specific but plausible token behavior (sender-surcharge) and its inclusion/selection as an output. Such tokens are uncommon but can appear; once present, failure is deterministic.

Exploitation

Exploitation Scenarios:

Scenario 1.
Per-asset DoS in atomic routes: A route deposits B units of a sender-surcharge ERC-20 into Deliver and immediately calls deliverToken with min = min, then safeTransfer(recipient, B) reverts because the token requires B + fee(B). The entire transaction reverts, making the asset persistently undeliverable via Deliver.
Preconditions / Assumptions
(a). The ERC-20 used as output has sender-surcharge semantics: transfer debits amount + fee(amount) > 0 from the sender for any positive amount.
(b). The route executes atomically: deposit into Deliver and then call deliverToken in the same transaction.
(c). min is chosen such that balance >= min.
(d). The asset is not excluded by upstream whitelisting/route selection.

Scenario 2.
Solver gas griefing: A sender-surcharge ERC-20 is not yet filtered. Solvers attempt to fulfill routes ending in Deliver.deliverToken for this asset; each attempt reverts as above, wasting gas and causing operational instability until filters are updated.
Preconditions / Assumptions
(a). The ERC-20 has sender-surcharge semantics and is not yet filtered by solvers.

(b). Solvers attempt execution (or retry variants) of routes ending in Deliver.deliverToken before fully blacklisting the asset.
(c). The asset is available to be selected as a terminal output in route construction.